

Documentación del Laboratorio 3: Integración de una Interfaz SPI y Sensor ADXL362 en un Sistema RISC-V sobre FPGA

Thomas Reed, Emanuel Varela, Andres Madrigal, Santiago Chavarria
Escuela de Ingeniería Electrónica
EL3313 Taller de Diseño Digital
Tecnológico de Costa Rica

Resumen—Este documento presenta el diseño e implementación de un sistema embebido basado en un procesador RISC-V RV32I sobre una FPGA Nexys4. El sistema integra una memoria ROM para almacenamiento del programa, una memoria RAM para manejo de datos y un conjunto de periféricos mapeados en memoria, incluyendo una interfaz UART para comunicación serial y una interfaz SPI para la comunicación con el acelerómetro ADXL362 incorporado en la tarjeta de desarrollo.

La arquitectura implementada utiliza el núcleo PicoRV32 junto con un controlador de bus encargado de administrar el acceso a memoria y periféricos. El módulo SPI desarrollado permite la lectura y escritura de registros del sensor ADXL362, posibilitando la adquisición de datos de aceleración en los ejes X, Y y Z. Adicionalmente, la interfaz UART facilita la transmisión de información hacia una computadora para tareas de monitoreo y depuración del sistema.

La validación del diseño se realizó mediante simulaciones funcionales y pruebas de integración de los distintos módulos de hardware descritos en SystemVerilog. Los resultados obtenidos demuestran el correcto funcionamiento de la arquitectura propuesta y la comunicación exitosa entre el procesador RISC-V y el sensor ADXL362 mediante el protocolo SPI.

Index Terms—RISC-V, PicoRV32, FPGA, Nexys4, SPI, ADXL362, UART, Sistemas Embebidos, SystemVerilog, Periféricos Mapeados en Memoria.

I. INTRODUCCIÓN

Los sistemas embebidos modernos requieren la integración eficiente de componentes de hardware y software para interactuar con dispositivos externos y procesar información en tiempo real. En este contexto, los procesadores basados en la arquitectura RISC-V han ganado relevancia debido a su naturaleza abierta, flexibilidad de implementación y amplia adopción en aplicaciones académicas e industriales.

El presente proyecto corresponde al Laboratorio 3 del curso EL3313 Taller de Diseño Digital, en el cual se desarrolla un sistema empotrado basado en un procesador RISC-V RV32I implementado sobre una FPGA Nexys4. Como extensión del sistema desarrollado en laboratorios anteriores, se incorpora una interfaz de comunicación serial SPI (Serial Peripheral Interface), permitiendo la interacción con el acelerómetro ADXL362 integrado en la tarjeta de desarrollo.

La arquitectura implementada está compuesta por el núcleo PicoRV32, una memoria ROM destinada al almacenamiento del programa, una memoria RAM para el manejo de datos temporales, un controlador de bus encargado de la administración

del mapa de memoria y un conjunto de periféricos mapeados en memoria. Entre estos periféricos se encuentran una interfaz UART para la comunicación serial con una computadora, módulos de entrada y salida para switches y LEDs, y un controlador SPI encargado de la configuración y adquisición de datos provenientes del acelerómetro.

El software del sistema fue desarrollado en lenguaje ensamblador para la arquitectura RV32I y permite inicializar el sensor ADXL362, leer periódicamente las mediciones correspondientes a los ejes X, Y y Z, almacenar temporalmente los datos obtenidos y transmitir la información mediante la interfaz UART. De esta manera, el sistema integra conceptos de arquitectura de computadores, diseño digital, protocolos de comunicación serial y programación de bajo nivel dentro de una plataforma de hardware reconfigurable.

En este documento se presenta la arquitectura general del sistema desarrollado, la descripción de los módulos de hardware implementados, la estrategia utilizada para la comunicación con el sensor ADXL362 y las pruebas realizadas para verificar el correcto funcionamiento de la solución propuesta.

II. MARCO TEÓRICO

II-A. Arquitectura RISC-V

RISC-V es una arquitectura de conjunto de instrucciones (ISA) de código abierto basada en la filosofía *Reduced Instruction Set Computer* (RISC). Su diseño modular permite implementar únicamente los componentes necesarios para una aplicación específica, reduciendo la complejidad del hardware y facilitando su utilización en sistemas embebidos.

En este proyecto se emplea el núcleo PicoRV32, una implementación ligera de la arquitectura RV32I diseñada para aplicaciones en FPGA. Este procesador ejecuta instrucciones almacenadas en memoria ROM y accede a memoria RAM y periféricos mediante un esquema de memoria mapeada.

II-B. Sistemas Embebidos sobre FPGA

Una FPGA (*Field Programmable Gate Array*) es un dispositivo lógico programable que permite implementar circuitos digitales complejos mediante lenguajes de descripción de hardware como SystemVerilog. A diferencia de los microcontroladores tradicionales, las FPGA permiten personalizar

completamente la arquitectura del sistema, incluyendo procesadores, memorias, buses y periféricos.

La tarjeta Nexys4 utilizada en este proyecto incorpora una FPGA de la familia Artix-7 y diversos periféricos integrados, entre ellos un acelerómetro ADXL362, LEDs, switches y una interfaz de comunicación serial.

II-C. Protocolo SPI

El protocolo SPI (*Serial Peripheral Interface*) es un mecanismo de comunicación serial síncrona ampliamente utilizado para la transferencia de datos entre dispositivos electrónicos. Este protocolo emplea una arquitectura maestro-esclavo donde un dispositivo maestro controla la comunicación mediante una señal de reloj.

Las señales principales del protocolo SPI son:

- **SCLK**: señal de reloj generada por el maestro.
- **MOSI**: línea de datos desde el maestro hacia el esclavo.
- **MISO**: línea de datos desde el esclavo hacia el maestro.
- **CS**: señal de selección del dispositivo esclavo.

En el sistema desarrollado, el procesador accede al acelerómetro ADXL362 mediante un controlador SPI implementado en hardware, permitiendo la lectura y escritura de registros internos del sensor.

II-D. Comunicación UART

UART (*Universal Asynchronous Receiver Transmitter*) es un protocolo de comunicación serial asíncrona utilizado para intercambiar información entre dispositivos digitales. La transmisión se realiza mediante una línea de datos y no requiere una señal de reloj compartida entre transmisor y receptor.

En este proyecto la interfaz UART se utiliza para transmitir información desde la FPGA hacia una computadora, facilitando la visualización de los datos obtenidos por el acelerómetro y las tareas de depuración del sistema.

II-E. Sensor ADXL362

El ADXL362 es un acelerómetro digital de tres ejes y bajo consumo energético fabricado por Analog Devices. El dispositivo permite medir aceleraciones en los ejes X, Y y Z y proporciona acceso a sus registros internos mediante una interfaz SPI.

Para su operación es necesario configurar registros de control relacionados con el modo de funcionamiento del sensor. Posteriormente, las mediciones de aceleración pueden obtenerse mediante la lectura de registros específicos correspondientes a cada eje.

En este proyecto el ADXL362 se configura desde el procesador RISC-V mediante transacciones SPI, permitiendo la adquisición periódica de datos de aceleración que posteriormente son transmitidos a través de la interfaz UART.

III. DESCRIPCIÓN DEL SISTEMA

El sistema implementado corresponde a una arquitectura embebida basada en un procesador RISC-V RV32I sobre una FPGA Nexys4. La arquitectura integra una memoria ROM para el almacenamiento del programa, una memoria RAM para

datos temporales, un controlador de bus para la decodificación de direcciones y un conjunto de periféricos mapeados en memoria.

Entre los periféricos principales se incluyen una interfaz UART para comunicación serial, registros de entrada y salida para switches y LEDs, y una interfaz SPI utilizada para la comunicación con el acelerómetro ADXL362. La organización general del sistema se muestra en la Figura 1.

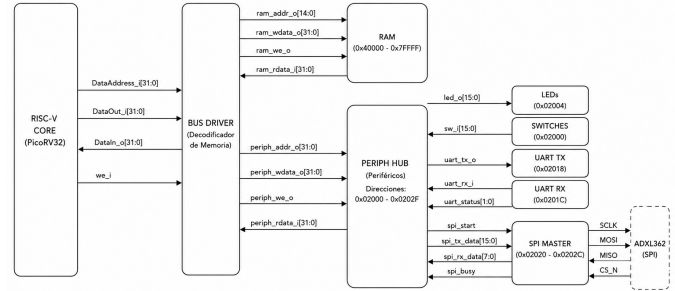


Figura 1: Diagrama de arquitectura del sistema RISC-V con periféricos mapeados en memoria y comunicación SPI con el ADXL362.

El acceso a los distintos recursos del sistema se realiza mediante un mapa de memoria. El procesador puede acceder a la RAM de datos y a los periféricos utilizando direcciones específicas. Los periféricos se ubican en el rango de direcciones 0x02000 a 0x0202C, mientras que la RAM de datos se encuentra a partir de la dirección 0x40000. La Figura 2 muestra la distribución utilizada.

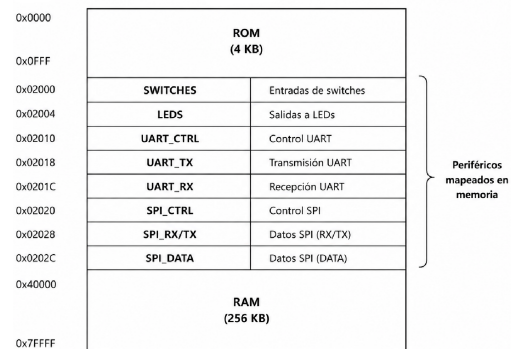


Figura 2: Mapa de memoria del sistema implementado.

El módulo PERIPH_HUB concentra la lógica asociada a los periféricos del sistema. Este bloque recibe las señales de dirección, datos de escritura y habilitación de escritura provenientes del controlador de bus, y se encarga de activar el periférico correspondiente según la dirección solicitada. Además, multiplexa los datos de lectura para devolver al procesador la información proveniente de switches, UART o SPI. La Figura 7 muestra sus principales entradas y salidas.

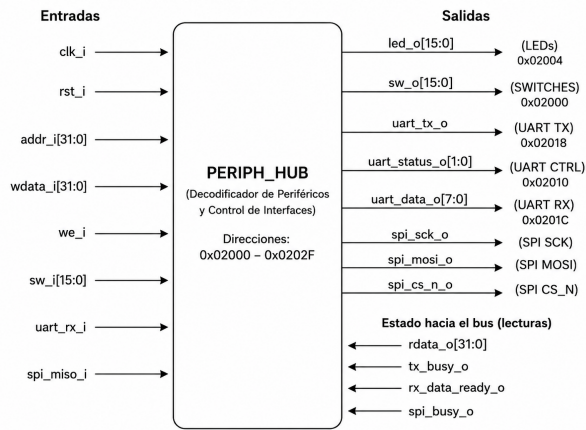


Figura 3: Diagrama de entradas y salidas del módulo PERIPH_HUB.

Dentro del bloque de periféricos se encuentra la interfaz asociada al controlador SPI. Esta sección del sistema permite que el procesador configure una transacción SPI mediante registros mapeados en memoria. El registro de control se ubica en la dirección 0x02020, mientras que los registros de datos se ubican en 0x02028 y 0x0202C. La Figura ?? muestra la interfaz funcional del módulo SPI desde el punto de vista del bus del procesador.

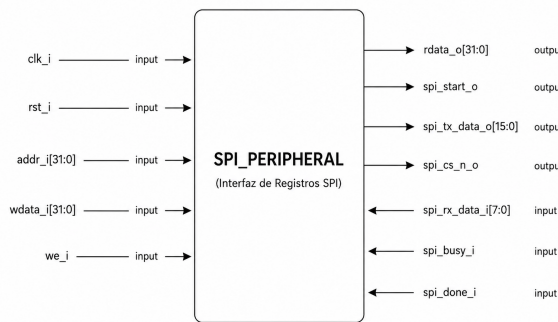


Figura 4: Diagrama de entradas y salidas del módulo SPI_PERIPHERAL.

El módulo SPI_MASTER es el encargado de generar las señales físicas del protocolo SPI hacia el acelerómetro ADXL362. Este bloque recibe los registros de control y transmisión, ejecuta la secuencia de transferencia serial y genera las señales spi_sck_o, spi_mosi_o y spi_cs_n_o. Asimismo, recibe la señal spi_miso_i proveniente del sensor y almacena los datos leídos en registros disponibles para el procesador. La Figura ?? muestra su organización funcional.

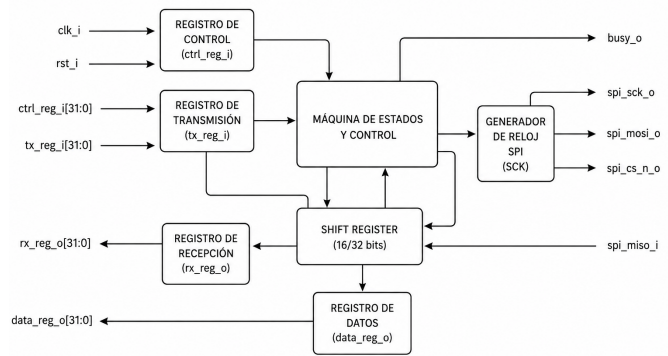


Figura 5: Diagrama de arquitectura SPI digital del módulo SPI_MASTER.

Finalmente, el comportamiento del programa en ensamblador sigue una secuencia de inicialización y lectura periódica del acelerómetro. Primero se inicializan los periféricos del sistema, luego se configura el ADXL362 mediante sus registros internos, se verifica la comunicación leyendo el registro DEVID y posteriormente se ingresa en un ciclo principal donde se leen los datos de los ejes X, Y y Z. Estos datos son transmitidos mediante UART y el proceso se repite indefinidamente. La Figura ?? resume este comportamiento.

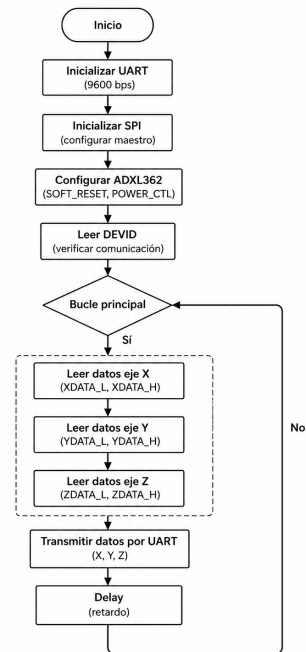


Figura 6: Diagrama de flujo del programa de adquisición de datos del ADXL362.

IV. IMPLEMENTACIÓN DEL SISTEMA

IV-A. Procesador RISC-V

El núcleo de procesamiento del sistema está basado en el procesador PicoRV32, una implementación ligera de la arquitectura RISC-V RV32I optimizada para aplicaciones en FPGA. Este procesador es el encargado de ejecutar el programa almacenado en memoria ROM y coordinar el acceso a los distintos recursos del sistema.

Para facilitar la integración con el resto de la arquitectura se desarrolló el módulo `riscv_core_wrapper`, el cual encapsula la interfaz de memoria unificada utilizada por PicoRV32 y la adapta a una arquitectura con memoria de programa y memoria de datos separadas. De esta manera, las instrucciones son obtenidas desde la memoria ROM mediante la interfaz de programa, mientras que las operaciones de lectura y escritura de datos son dirigidas hacia el controlador de bus.

El procesador utiliza una latencia de memoria de un ciclo de reloj. Cuando se realiza una solicitud de acceso, la dirección se presenta inmediatamente a la memoria o periférico correspondiente y la respuesta es retornada al procesador durante el siguiente ciclo de reloj. Este mecanismo simplifica la integración con los distintos módulos del sistema y mantiene una arquitectura completamente síncrona.

Las señales principales utilizadas para la comunicación con el resto del sistema son la dirección de programa (`ProgAddress_o`), la dirección de datos (`DataAddress_o`), el bus de escritura (`DataOut_o`), el bus de lectura (`DataIn_i`) y la señal de habilitación de escritura (`we_o`).

IV-B. Controlador de Bus

La comunicación entre el procesador RISC-V, la memoria RAM y los periféricos se realiza mediante el módulo `bus_driver`. Este bloque actúa como un decodificador de direcciones encargado de determinar el destino de cada operación de lectura o escritura solicitada por el procesador.

El controlador recibe las señales de dirección (`DataAddress_i`), datos de escritura (`DataOut_i`) y habilitación de escritura (`we_i`) provenientes del núcleo de procesamiento. A partir de la dirección solicitada, el módulo identifica si el acceso corresponde a la memoria RAM o a alguno de los periféricos mapeados en memoria.

La región de periféricos se encuentra ubicada en el rango de direcciones comprendido entre `0x02000` y `0x0202F`. Cuando la dirección pertenece a este rango, el controlador habilita la interfaz del módulo `PERIPH_HUB`, permitiendo el acceso a switches, LEDs, UART y SPI.

Por otra parte, la memoria RAM ocupa el rango de direcciones comprendido entre `0x40000` y `0x7FFFF`. Cuando la dirección corresponde a esta región, el controlador activa la interfaz de memoria y redirige las operaciones de lectura y escritura hacia la RAM de datos.

Para las operaciones de lectura, el controlador utiliza un multiplexor que selecciona la fuente de datos apropiada según

el dispositivo que respondió a la solicitud. De esta manera, los datos provenientes de la memoria RAM o de los periféricos son devueltos al procesador mediante el bus de lectura.

Esta organización permite implementar un esquema de periféricos mapeados en memoria, simplificando el acceso desde el software y permitiendo que todos los dispositivos del sistema sean controlados mediante instrucciones estándar de carga y almacenamiento de la arquitectura RISC-V.

IV-C. Peripheral Hub

El módulo `PERIPH_HUB` constituye el punto central de acceso a los periféricos implementados en el sistema. Su función principal consiste en recibir las solicitudes provenientes del controlador de bus, decodificar la dirección solicitada y dirigir la operación hacia el periférico correspondiente.

La Figura 7 muestra las principales entradas y salidas del módulo. Entre las señales de entrada se encuentran la dirección de acceso (`addr_i`), los datos de escritura (`wdata_i`), la señal de habilitación de escritura (`we_i`), las entradas de los switches (`sw_i`), la línea de recepción UART (`uart_rx_i`) y la línea de recepción SPI (`spi_miso_i`). Como salidas principales se generan las señales de control para LEDs, UART y SPI, además del bus de datos de lectura (`rdata_o`).

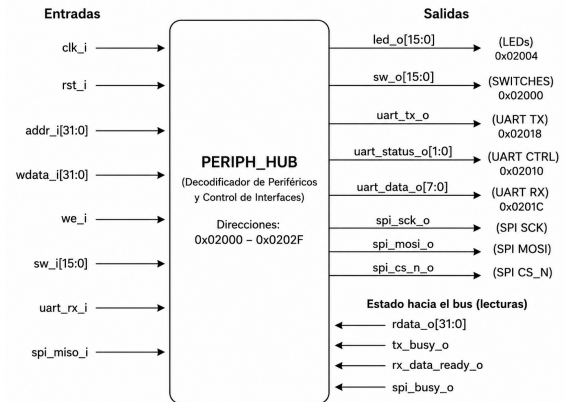


Figura 7: Diagrama de entradas y salidas del módulo `PERIPH_HUB`.

El módulo integra cuatro grupos principales de periféricos:

- **Switches:** permiten la lectura de las entradas físicas de la FPGA. Para mejorar la confiabilidad de las lecturas se implementa un mecanismo de sincronización y anti-rebote (*debouncing*), garantizando que únicamente los cambios estables sean reportados al procesador.
- **LEDs:** permiten visualizar información de estado mediante escritura sobre registros mapeados en memoria. El procesador puede controlar directamente el estado de los dieciséis LEDs disponibles en la tarjeta.
- **UART:** proporciona comunicación serial asíncrona con dispositivos externos. El módulo integra tanto el transmisor (`uart_tx`) como el receptor (`uart_rx`), permi-

tiendo enviar y recibir datos mediante registros mapeados en memoria.

- **SPI:** proporciona la interfaz de acceso al acelerómetro ADXL362. Mediante registros de control y datos, el procesador puede iniciar transacciones SPI, transmitir comandos y recuperar la información recibida desde el sensor. ""

El acceso a estos periféricos se realiza mediante un esquema de memoria mapeada. Cuando el procesador realiza una lectura, el módulo selecciona el dato correspondiente según la dirección solicitada y lo coloca sobre el bus de retorno. De forma similar, las operaciones de escritura actualizan los registros internos del periférico seleccionado.

Esta estrategia permite que todos los dispositivos de entrada y salida sean controlados mediante instrucciones estándar de lectura y escritura, simplificando el desarrollo del software y manteniendo una arquitectura uniforme para el acceso a los recursos del sistema.

IV-D. Interfaz SPI

La comunicación entre el procesador RISC-V y el módulo encargado de implementar el protocolo SPI se realiza mediante una interfaz basada en registros mapeados en memoria. Esta interfaz permite que el software controle las transferencias SPI utilizando operaciones convencionales de lectura y escritura sobre direcciones específicas del espacio de memoria.

La Figura 9 muestra la organización funcional del módulo SPI_PERIPHERAL, el cual actúa como intermediario entre el procesador y el controlador SPI implementado en hardware.

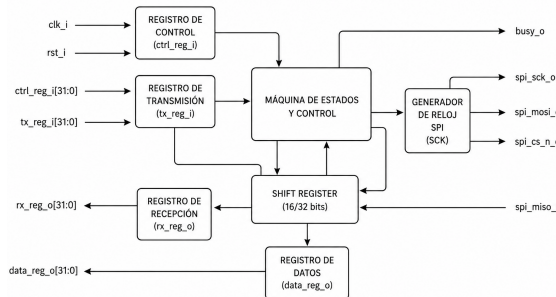


Figura 8: Diagrama de entradas y salidas del módulo SPI_PERIPHERAL.

El módulo recibe las señales de dirección, datos de escritura y habilitación de escritura provenientes del controlador de bus. A partir de estas señales se actualizan los registros internos asociados al controlador SPI y se generan las señales necesarias para iniciar una transacción.

Los registros utilizados para la comunicación SPI se encuentran ubicados en las siguientes direcciones:

Tabla I: Registros de la interfaz SPI

Dirección	Función
0x02020	Registro de control SPI (SPI_CTRL)
0x02028	Registro de transmisión y recepción (SPI_RX/TX)
0x0202C	Registro de datos (SPI_DATA)

El registro SPI_CTRL permite configurar la operación que realizará el controlador SPI. Mediante este registro se selecciona el tipo de transacción y se genera la señal de inicio correspondiente. El módulo detecta la escritura sobre este registro y produce un pulso de activación que inicia la transferencia SPI.

El registro SPI_RX/TX almacena los datos que serán transmitidos hacia el dispositivo esclavo. Durante una operación de lectura, este mismo registro permite recuperar la información recibida por el controlador SPI.

Por su parte, el registro SPI_DATA contiene los datos finales obtenidos durante una transacción y facilita su acceso desde el software ejecutado por el procesador.

Mediante esta organización, el software puede realizar operaciones de lectura y escritura sobre el acelerómetro ADXL362 sin necesidad de interactuar directamente con las señales físicas del protocolo SPI. La complejidad asociada a la generación del reloj serial, sincronización de datos y control de la transferencia queda encapsulada dentro del módulo SPI_MASTER, simplificando considerablemente el desarrollo del programa de aplicación.

IV-E. Módulo SPI Master

El módulo SPI_MASTER es el encargado de implementar el protocolo SPI a nivel de señales físicas. Su función principal consiste en generar las secuencias de comunicación necesarias para establecer transferencias de datos entre el procesador RISC-V y el acelerómetro ADXL362.

La arquitectura funcional del módulo se muestra en la Figura 9. Este bloque recibe comandos y datos provenientes de la interfaz SPI, ejecuta la transferencia serial y devuelve la información recibida al procesador mediante registros internos.

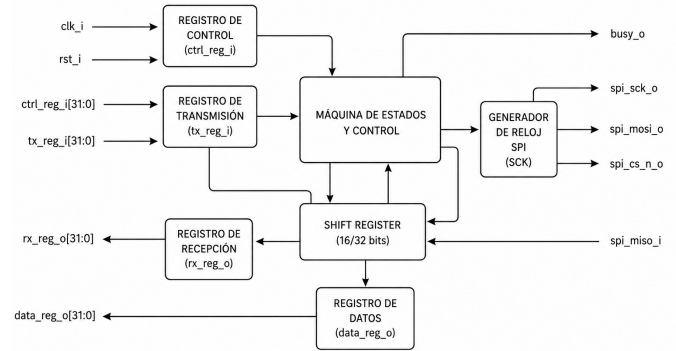


Figura 9: Diagrama de arquitectura SPI digital del módulo SPI_MASTER.

La operación del módulo se basa en varios bloques funcionales interconectados. El registro de control recibe la información necesaria para determinar el tipo de transacción a realizar, mientras que el registro de transmisión almacena los datos que serán enviados hacia el dispositivo esclavo.

Una máquina de estados finitos (*Finite State Machine*, FSM) coordina el proceso completo de transferencia. Este bloque supervisa el inicio de la comunicación, controla el desplazamiento de bits, administra la señal de selección del dispositivo (CS) y determina cuándo una transferencia ha finalizado. Asimismo, genera la señal `busy_o`, la cual informa al procesador que el controlador SPI se encuentra ocupado ejecutando una transacción.

El envío y recepción de información se realiza mediante un registro de desplazamiento (*Shift Register*). Durante cada ciclo de reloj SPI, un bit es transmitido a través de la línea MOSI, mientras que simultáneamente se captura un bit proveniente de la línea MISO. Este mecanismo permite aprovechar la naturaleza full-duplex del protocolo SPI.

El generador de reloj SPI produce la señal `spi_sck_o` utilizada para sincronizar la transferencia de datos. La frecuencia de esta señal se obtiene a partir del reloj principal del sistema mediante un divisor configurable, garantizando una operación compatible con las especificaciones del acelerómetro ADXL362.

Las señales físicas generadas por el módulo corresponden a:

- `spi_sck_o`: reloj serial utilizado para sincronizar la transferencia.
- `spi_mosi_o`: línea de datos transmitidos desde el maestro hacia el esclavo.
- `spi_miso_i`: línea de datos recibidos desde el esclavo hacia el maestro.
- `spi_cs_n_o`: señal de selección activa en bajo del dispositivo esclavo.

Una vez finalizada la transferencia, los datos recibidos son almacenados en los registros de recepción y datos, permitiendo que el procesador acceda a ellos mediante los registros mapeados en memoria definidos para la interfaz SPI.

Gracias a esta arquitectura, el software ejecutado por el procesador puede realizar operaciones de lectura y escritura sobre el acelerómetro ADXL362 utilizando únicamente accesos a memoria, mientras que toda la complejidad asociada al protocolo SPI es gestionada internamente por el controlador hardware.

IV-F. Interfaz UART

La comunicación serial entre la FPGA y la computadora se realiza mediante una interfaz UART (*Universal Asynchronous Receiver Transmitter*). Este periférico permite transmitir y recibir datos de forma asíncrona utilizando únicamente las líneas de transmisión (TX) y recepción (RX), sin necesidad de una señal de reloj compartida entre los dispositivos.

La implementación desarrollada incorpora módulos independientes para transmisión y recepción, denominados `uart_tx` y `uart_rx`, respectivamente. Ambos módulos son

integrados dentro del bloque `PERIPH_HUB` y son accesibles desde el procesador mediante registros mapeados en memoria.

Los registros asociados a la interfaz UART se muestran en la Tabla II.

Tabla II: Registros de la interfaz UART

Dirección	Función
0x02010	Registro de control UART (UART_CTRL)
0x02018	Registro de transmisión (UART_TX)
0x0201C	Registro de recepción (UART_RX)

El registro `UART_CTRL` permite supervisar el estado de la interfaz. Entre los indicadores disponibles se encuentran el estado del transmisor (TX Busy) y la presencia de datos recibidos (RX Data Ready). Estos indicadores permiten que el software determine cuándo es posible transmitir un nuevo dato o cuándo existe información disponible para lectura.

Cuando el procesador escribe un byte en el registro `UART_TX`, el módulo transmisor inicia automáticamente la generación de la trama serial correspondiente. La transmisión se realiza utilizando el formato estándar 8N1, el cual consiste en un bit de inicio, ocho bits de datos, ausencia de bit de paridad y un bit de parada.

Por otra parte, el módulo receptor monitorea continuamente la línea serial de entrada. Cuando se detecta una trama válida, el byte recibido es almacenado en el registro `UART_RX` y se activa el indicador correspondiente en el registro de control para notificar al procesador la disponibilidad de nuevos datos.

Dentro de la aplicación desarrollada, la interfaz UART se utiliza principalmente para transmitir hacia una computadora los datos obtenidos desde el acelerómetro ADXL362. Esta funcionalidad facilita la visualización de las mediciones y permite verificar el correcto funcionamiento de la comunicación SPI y del sistema completo durante las pruebas experimentales.

V. IMPLEMENTACIÓN DEL SOFTWARE

El software del sistema fue desarrollado en lenguaje ensamblador para la arquitectura RV32I y almacenado en la memoria ROM del procesador. Su función principal consiste en inicializar el acelerómetro ADXL362, adquirir periódicamente las mediciones de aceleración y transmitir los resultados mediante la interfaz UART.

V-A. Inicialización del Sistema

Al iniciar la ejecución, el programa configura la pila de ejecución utilizando la memoria RAM disponible y establece las direcciones base necesarias para acceder a los periféricos UART, SPI y RAM.

Como indicador visual de arranque, el software escribe el patrón hexadecimal `0xAAAA` en el registro de LEDs. Posteriormente, se transmite el mensaje `ADXL` mediante UART con el objetivo de confirmar que el sistema ha iniciado correctamente y que la comunicación serial se encuentra operativa.

V-B. Configuración del ADXL362

La inicialización del acelerómetro se realiza mediante la rutina `adxl_init`. Esta rutina utiliza la interfaz SPI para escribir sobre los registros de configuración del sensor.

Inicialmente se ejecuta un reinicio interno del dispositivo escribiendo el valor `0x52` en el registro `SOFT_RESET` (`0x1F`). Posteriormente se configura el registro `POWER_CTL` (`0x2D`) con el valor `0x02`, habilitando el modo de medición del acelerómetro.

Una vez finalizada la configuración, el programa realiza la lectura del registro `DEVID` para verificar el correcto funcionamiento de la comunicación SPI y confirmar la presencia del dispositivo.

V-C. Lectura de Datos del Acelerómetro

La adquisición de datos se realiza mediante la rutina `adxl_read_xyz`. Esta función efectúa múltiples lecturas SPI para recuperar los registros correspondientes a las componentes de aceleración de los tres ejes.

Los registros utilizados son:

- `0x0E` y `0x0F` para el eje X.
- `0x10` y `0x11` para el eje Y.
- `0x12` y `0x13` para el eje Z.

Cada medición se encuentra distribuida en dos bytes consecutivos. El software combina ambos valores para formar una palabra de 16 bits y posteriormente realiza una extensión de signo para obtener el valor entero correspondiente.

Los resultados finales son almacenados en los registros `s0`, `s1` y `s2`, correspondientes a las aceleraciones medidas en los ejes X, Y y Z, respectivamente.

V-D. Almacenamiento en Memoria RAM

Después de cada adquisición, las mediciones obtenidas son almacenadas temporalmente en la memoria RAM del sistema. Los valores correspondientes a los ejes X, Y y Z son escritos en posiciones consecutivas a partir de la dirección base de la memoria de datos.

Este mecanismo permite conservar las últimas muestras adquiridas y facilita su acceso desde otras rutinas de software o futuras ampliaciones del sistema.

V-E. Transmisión de Datos mediante UART

La visualización de resultados se realiza utilizando la interfaz UART. Para cada ciclo de adquisición, el programa transmite una cadena de caracteres con el siguiente formato:

`X=<valor>,Y=<valor>,Z=<valor>`

La rutina `print_int` convierte cada medición numérica a su representación ASCII antes de transmitirla. Asimismo, las funciones `uart_tx` y `newline` son utilizadas para gestionar el envío de caracteres y el formato de salida.

V-F. Bucle Principal de Ejecución

Una vez completada la inicialización del sistema, el programa entra en un ciclo infinito de ejecución. Durante cada iteración se realizan las siguientes operaciones:

1. Lectura de las mediciones X, Y y Z del acelerómetro.
2. Almacenamiento temporal de los datos en memoria RAM.
3. Conversión de los valores a formato ASCII.
4. Transmisión de los resultados mediante UART.
5. Espera aproximada de 10 ms.

Este proceso se repite indefinidamente, permitiendo la adquisición continua de datos del acelerómetro y su monitoreo en tiempo real desde una computadora conectada al puerto serial.

VI. APLICACIÓN INTERACTIVA: PIANO VIRTUAL

Con el objetivo de demostrar la funcionalidad del sistema desarrollado, se implementó una aplicación interactiva en Python que utiliza los datos provenientes del acelerómetro ADXL362 como mecanismo de control en tiempo real. La aplicación consiste en un piano virtual capaz de generar notas musicales mediante el teclado de la computadora y modificar distintos efectos de audio utilizando la información adquirida por la FPGA.

VI-A. Arquitectura General

La arquitectura completa del sistema se encuentra compuesta por tres niveles funcionales. En el primer nivel, el acelerómetro ADXL362 proporciona las mediciones de aceleración en los ejes X, Y y Z. Estas mediciones son obtenidas por el procesador RISC-V mediante la interfaz SPI implementada en hardware. Posteriormente, los datos son transmitidos hacia una computadora utilizando la interfaz UART.

En el segundo nivel, una aplicación desarrollada en Python recibe continuamente los datos enviados por la FPGA a través del puerto serial. Finalmente, en el tercer nivel, dichos datos son utilizados para modificar parámetros de un sintetizador digital encargado de generar las notas musicales del piano virtual.

VI-B. Comunicación UART

La aplicación implementa una interfaz serial basada en la librería `PySerial`. Los datos enviados por el procesador son recibidos en formato ASCII, permitiendo la transferencia de las componentes de aceleración correspondientes a los ejes X, Y y Z.

Una vez recibidos, los valores son procesados y convertidos a variables internas utilizadas por el motor de síntesis de audio. Este mecanismo permite establecer una comunicación continua entre la FPGA y la aplicación de usuario ejecutada en la computadora.

VI-C. Generación de Audio

La generación de sonido se realiza mediante un sintetizador digital implementado en Python. El sistema utiliza osciladores senoidales para producir las distintas notas musicales y permite la ejecución simultánea de múltiples teclas, proporcionando capacidad de polifonía.

Las frecuencias de cada nota son calculadas automáticamente a partir de la escala musical temperada, permitiendo generar sonidos comprendidos entre las notas C3 y C5. El usuario puede interpretar melodías utilizando el teclado de la computadora como interfaz principal de entrada.

VI-D. Control Mediante el Acelerómetro

La principal característica de la aplicación consiste en utilizar las mediciones del acelerómetro para modificar dinámicamente el comportamiento del sintetizador.

Durante la ejecución, la aplicación realiza una calibración inicial para determinar la posición de referencia del sensor. A partir de dicha referencia se calcula la magnitud del movimiento detectado y la variación experimentada por cada eje.

La intensidad del movimiento es utilizada para controlar el nivel de distorsión aplicado al sonido generado, mientras que la inclinación sobre uno de los ejes permite implementar un efecto de *pitch bend*, modificando la frecuencia de las notas reproducidas en tiempo real.

De esta forma, el acelerómetro se convierte en un controlador gestual capaz de alterar las características del sonido mediante movimientos físicos realizados por el usuario.

VI-E. Funcionamiento General

El flujo de operación de la aplicación puede resumirse de la siguiente manera:

1. El acelerómetro ADXL362 mide las aceleraciones en los ejes X, Y y Z.
2. El procesador RISC-V obtiene los datos mediante la interfaz SPI.
3. Las mediciones son transmitidas hacia la computadora mediante UART.
4. La aplicación Python recibe y procesa la información recibida.
5. El sintetizador genera las notas musicales correspondientes.
6. Los movimientos detectados por el acelerómetro modifican la distorsión y el *pitch bend* del sonido generado.

Esta aplicación demuestra la integración completa entre hardware digital, procesamiento embebido y software de alto nivel, permitiendo utilizar el acelerómetro como un dispositivo de control para una aplicación interactiva en tiempo real.

VII. CONCLUSIONES

1. Se logró implementar exitosamente una interfaz SPI en hardware utilizando SystemVerilog, permitiendo la comunicación entre el procesador RISC-V y el acelerómetro ADXL362. La arquitectura desarrollada permitió

realizar operaciones de lectura y escritura sobre los registros del sensor, validando el correcto funcionamiento del protocolo SPI dentro de la FPGA.

2. La integración de los módulos de hardware, incluyendo el procesador PicoRV32, el controlador de bus, la interfaz UART y el controlador SPI, permitió construir un sistema embebido funcional capaz de adquirir datos del acelerómetro y transmitirlos hacia una aplicación externa en tiempo real.
3. El uso de una aplicación desarrollada en Python demostró la posibilidad de utilizar los datos adquiridos por el acelerómetro como mecanismo de control para aplicaciones interactivas. En este proyecto, las mediciones de aceleración fueron empleadas para modificar parámetros de un piano virtual, incorporando efectos de distorsión y *pitch bend* controlados mediante movimiento.
4. El desarrollo realizado permitió integrar conocimientos relacionados con diseño digital, sistemas embebidos, protocolos de comunicación y programación de aplicaciones, evidenciando la importancia de la interacción entre hardware y software en la implementación de sistemas electrónicos modernos.
5. Como trabajo futuro, podrían incorporarse interfaces gráficas más avanzadas, nuevos efectos de audio o sensores adicionales, con el fin de ampliar las capacidades del sistema y explorar aplicaciones más complejas basadas en interacción gestual y procesamiento en tiempo real.